

ASSIGNMENT – 9.5

NAME: YEROJU ANJALI

HT.NO: 2303A51416

BT.NO: 29

LAB EXPERIMENT : Lab Experiment: Documentation Generation – Automatic documentation and code comments.

PROBLEM -01:

Task:

1. Write documentation in:

o (a) Docstring

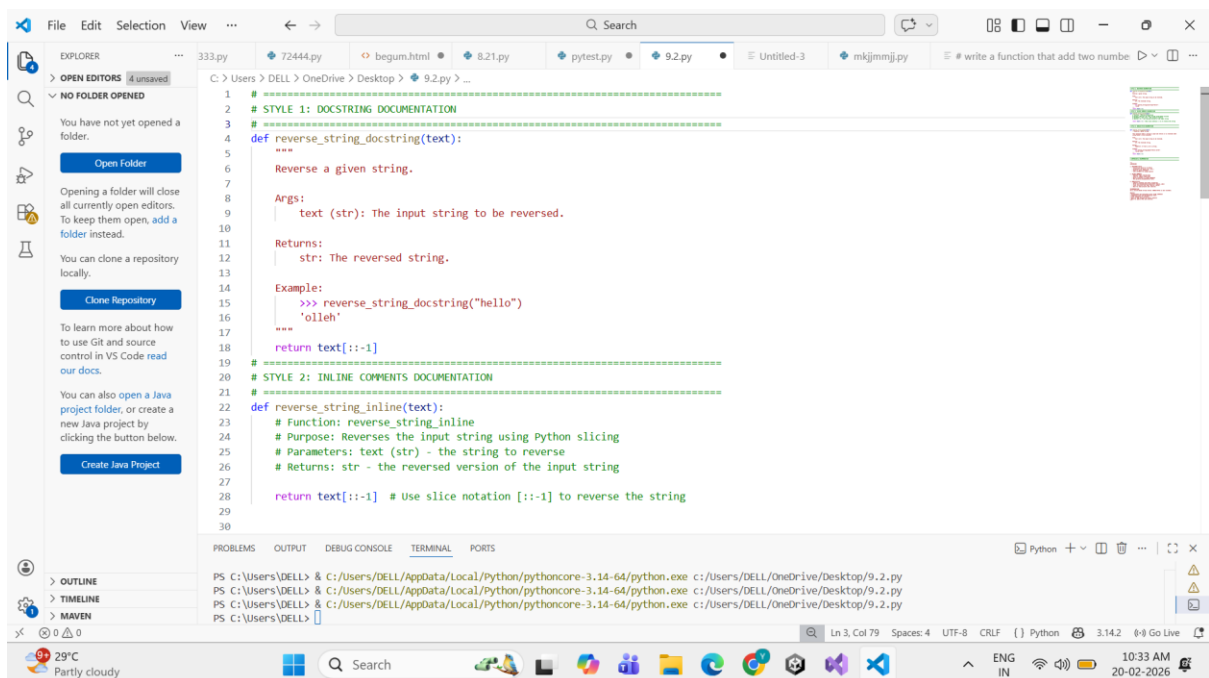
o (b) Inline comments

o (c) Google-style documentation

2. Compare the three documentation styles.

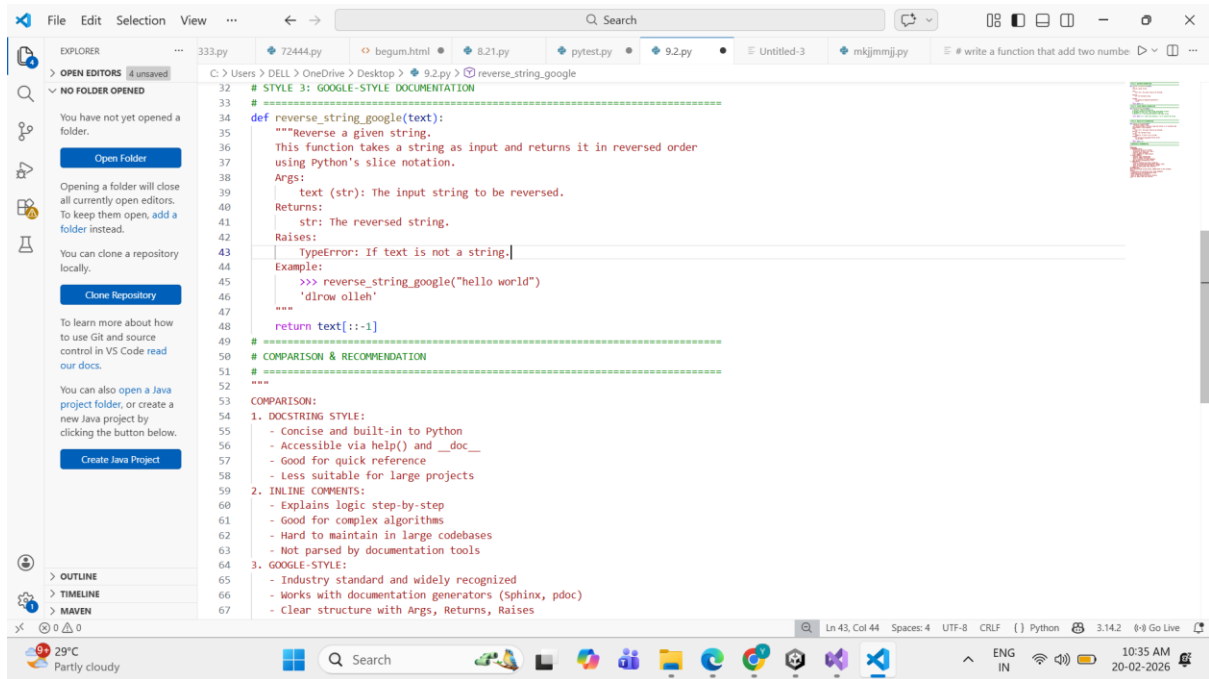
3. Recommend the most suitable style for a utility-based string library.

CODE:



```
1 # =====  
2 # STYLE 1: DOCSTRING DOCUMENTATION  
3 # =====  
4 def reverse_string_docstring(text):  
5     """  
6     Reverse a given string.  
7  
8     Args:  
9         text (str): The input string to be reversed.  
10  
11     Returns:  
12         str: The reversed string.  
13  
14     Example:  
15     >>> reverse_string_docstring("hello")  
16     'olleh'  
17     """  
18     return text[::-1]  
19 # =====  
20 # STYLE 2: INLINE COMMENTS DOCUMENTATION  
21 # =====  
22 def reverse_string_inline(text):  
23     # Function: reverse_string_inline  
24     # Purpose: Reverses the input string using Python slicing  
25     # Parameters: text (str) - the string to reverse  
26     # Returns: str - the reversed version of the input string  
27     return text[::-1] # Use slice notation [::-1] to reverse the string  
28  
29  
30
```

The screenshot shows the Visual Studio Code interface. The Explorer panel on the left shows a file named '9.2.py'. The main editor area displays the Python code with three different documentation styles. The first style uses a multi-line docstring. The second style uses inline comments for documentation. The third style uses a multi-line docstring in Google style. The bottom panel shows the terminal with the command 'python 9.2.py' and the output 'olleh'.



COMPARISON OF DOCUMENTATION STYLES :

1. Docstring Style
Concise and built into Python. Accessible using `help()` and `__doc__`. Good for small projects, but less structured for large systems.
2. Inline Comments
Explain code logic step-by-step. Useful for complex algorithms, but difficult to maintain in large codebases and not supported by documentation tools.
3. Google-Style Documentation
Structured and industry standard. Includes sections like Args and Returns. Works well with documentation tools and team projects.

RECOMMENDATION

For a utility-based string library, Google-style documentation is best because it is professional, structured, tool-friendly, and suitable for large or public projects.

Problem 2: Password Strength Checker

Consider the function:

```
def check_strength(password):

return len(password) >= 8
```

Task:

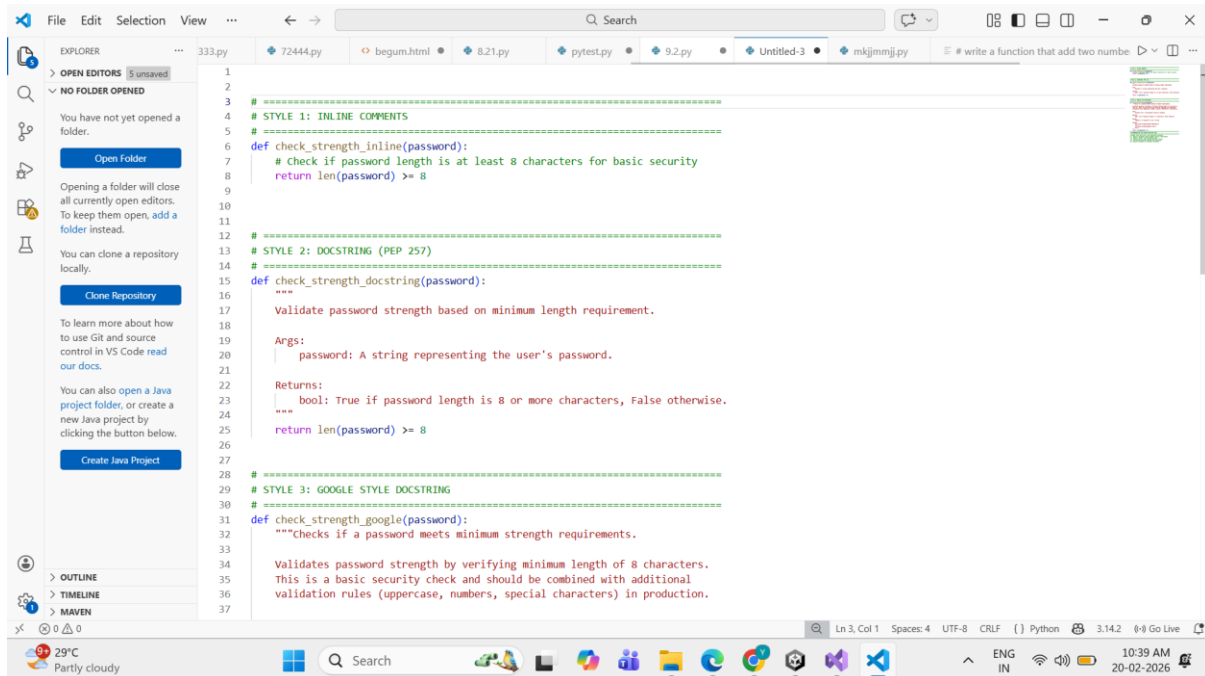
1. Document the function using docstring, inline comments, and

Google style.

2. Compare documentation styles for security-related code.

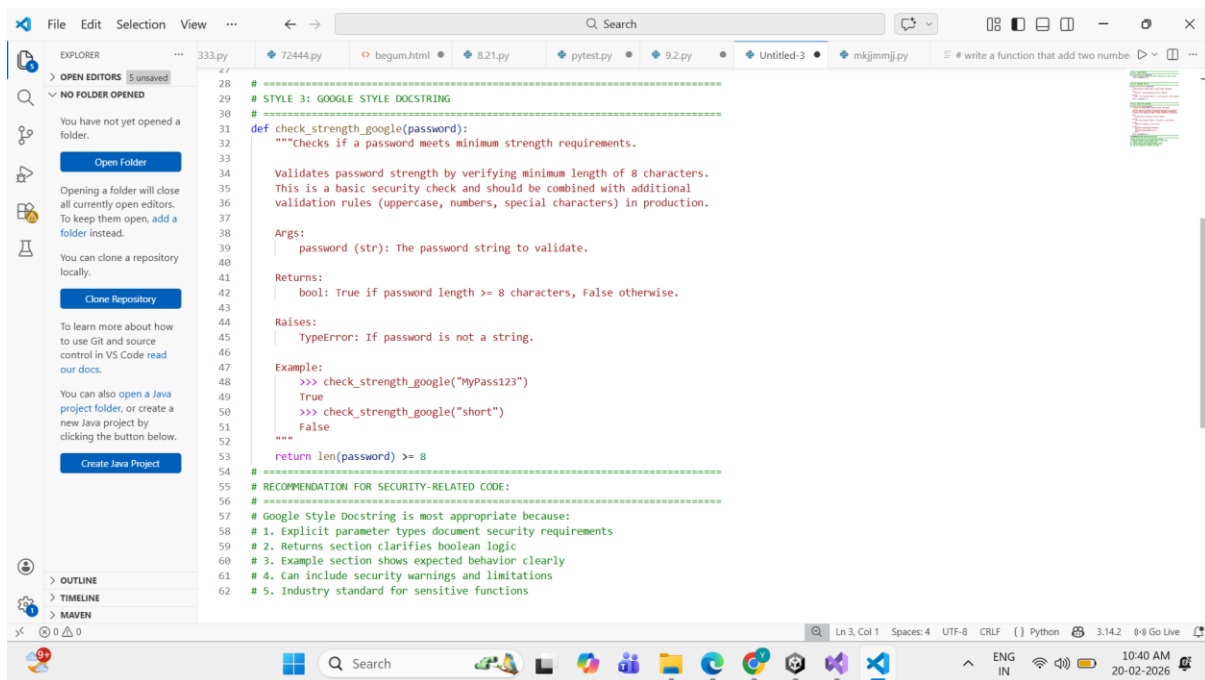
3. Recommend the most appropriate style.

CODE:



The screenshot shows the Visual Studio Code editor with a Python file named 333.py. The editor displays three different documentation styles for a function named `check_strength`. The Explorer sidebar on the left shows the file structure, and the Outline sidebar shows the function definition. The status bar at the bottom indicates the file is in Python mode, UTF-8 encoding, and CRLF line endings.

```
1
2
3 # =====
4 # STYLE 1: INLINE COMMENTS
5 # =====
6 def check_strength_inline(password):
7     # Check if password length is at least 8 characters for basic security
8     return len(password) >= 8
9
10
11
12 # =====
13 # STYLE 2: DOCSTRING (PEP 257)
14 # =====
15 def check_strength_docstring(password):
16     """
17     Validate password strength based on minimum length requirement.
18
19     Args:
20         password: A string representing the user's password.
21
22     Returns:
23         bool: True if password length is 8 or more characters, False otherwise.
24     """
25     return len(password) >= 8
26
27
28 # =====
29 # STYLE 3: GOOGLE STYLE DOCSTRING
30 # =====
31 def check_strength_google(password):
32     """Checks if a password meets minimum strength requirements.
33
34     Validates password strength by verifying minimum length of 8 characters.
35     This is a basic security check and should be combined with additional
36     validation rules (uppercase, numbers, special characters) in production.
37
38     Args:
39         password (str): The password string to validate.
40
41     Returns:
42         bool: True if password length >= 8 characters, False otherwise.
43
44     Raises:
45         TypeError: If password is not a string.
46
47     Example:
48         >>> check_strength_google("MyPass123")
49         True
50         >>> check_strength_google("short")
51         False
52     """
53     return len(password) >= 8
54
55 # =====
56 # RECOMMENDATION FOR SECURITY-RELATED CODE:
57 # =====
58 # Google Style Docstring is most appropriate because:
59 # 1. Explicit parameter types document security requirements
60 # 2. Returns section clarifies boolean logic
61 # 3. Example section shows expected behavior clearly
62 # 4. Can include security warnings and limitations
63 # 5. Industry standard for sensitive functions
```



This screenshot shows a closer view of the Google Style Docstring for the `check_strength_google` function. The function is defined with a docstring that includes a description, arguments, returns, raises, and an example. The status bar at the bottom indicates the file is in Python mode, UTF-8 encoding, and CRLF line endings.

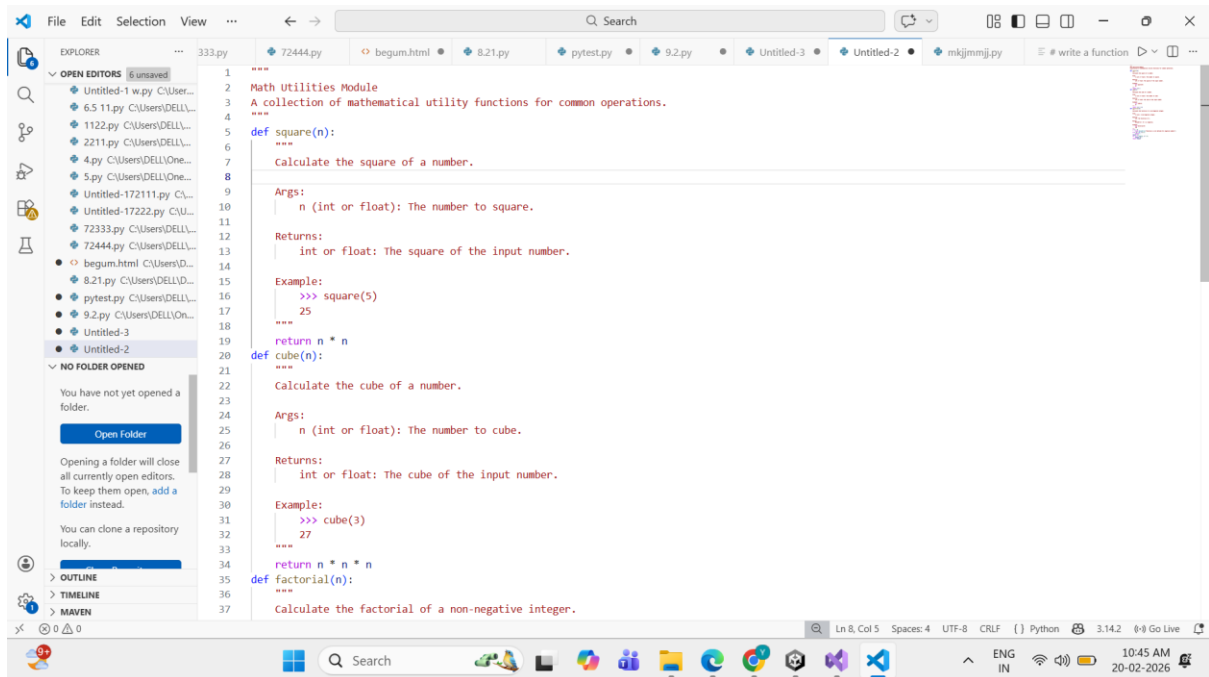
```
28
29 # =====
30 # STYLE 3: GOOGLE STYLE DOCSTRING
31 # =====
32 def check_strength_google(password):
33     """Checks if a password meets minimum strength requirements.
34
35     Validates password strength by verifying minimum length of 8 characters.
36     This is a basic security check and should be combined with additional
37     validation rules (uppercase, numbers, special characters) in production.
38
39     Args:
40         password (str): The password string to validate.
41
42     Returns:
43         bool: True if password length >= 8 characters, False otherwise.
44
45     Raises:
46         TypeError: If password is not a string.
47
48     Example:
49         >>> check_strength_google("MyPass123")
50         True
51         >>> check_strength_google("short")
52         False
53     """
54     return len(password) >= 8
55
56 # =====
57 # RECOMMENDATION FOR SECURITY-RELATED CODE:
58 # =====
59 # Google Style Docstring is most appropriate because:
60 # 1. Explicit parameter types document security requirements
61 # 2. Returns section clarifies boolean logic
62 # 3. Example section shows expected behavior clearly
63 # 4. Can include security warnings and limitations
64 # 5. Industry standard for sensitive functions
```

Problem 3: Math Utilities Module

Task:

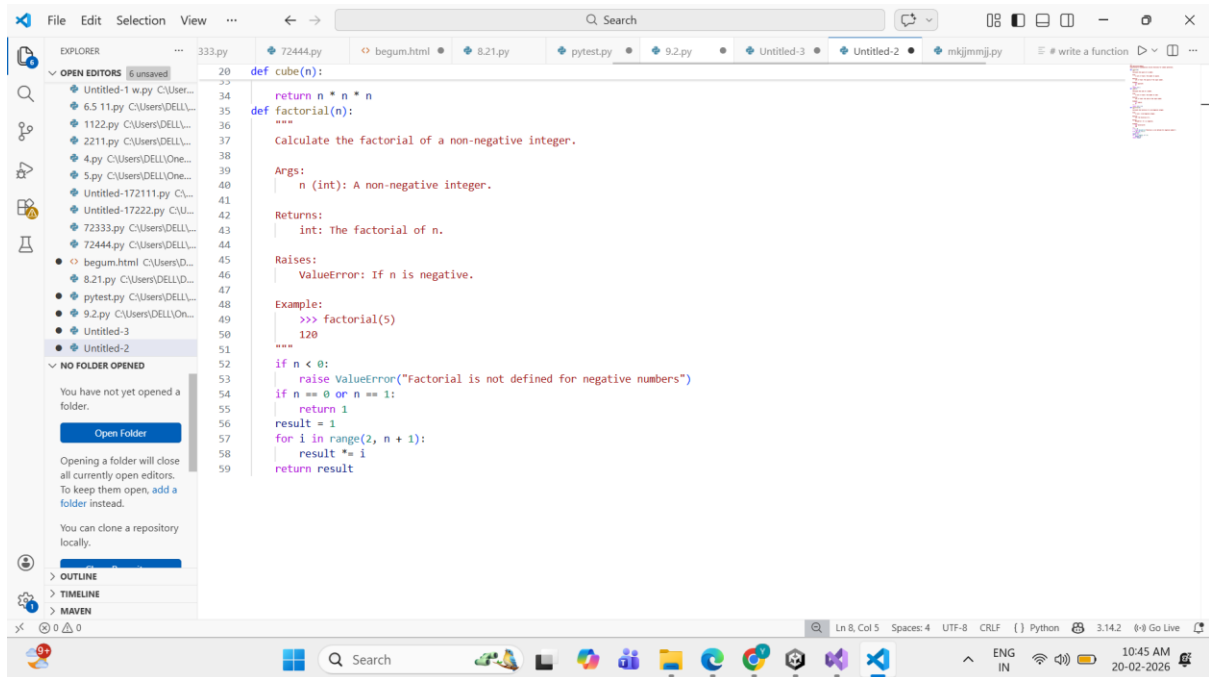
1. Create a module `math_utils.py` with functions:
 - o `square(n)`
 - o `cube(n)`
 - o `factorial(n)`
2. Generate docstrings automatically using AI tools.
3. Export documentation as an HTML file.

CODE:



The screenshot shows a code editor with the following content:

```
1  """
2  Math Utilities Module
3  A collection of mathematical utility functions for common operations.
4  """
5
6  def square(n):
7      """
8      Calculate the square of a number.
9
10     Args:
11         n (int or float): The number to square.
12
13     Returns:
14         int or float: The square of the input number.
15
16     Example:
17         >>> square(5)
18         25
19     """
20     return n * n
21
22  def cube(n):
23      """
24      Calculate the cube of a number.
25
26     Args:
27         n (int or float): The number to cube.
28
29     Returns:
30         int or float: The cube of the input number.
31
32     Example:
33         >>> cube(3)
34         27
35     """
36     return n * n * n
37
38  def factorial(n):
39      """
40      Calculate the factorial of a non-negative integer.
41
42     Args:
43         n (int): A non-negative integer.
44
45     Returns:
46         int: The factorial of the input number.
47
48     Example:
49         >>> factorial(5)
50         120
51     """
52     # Implementation of factorial function
53     result = 1
54     for i in range(1, n + 1):
55         result *= i
56     return result
```

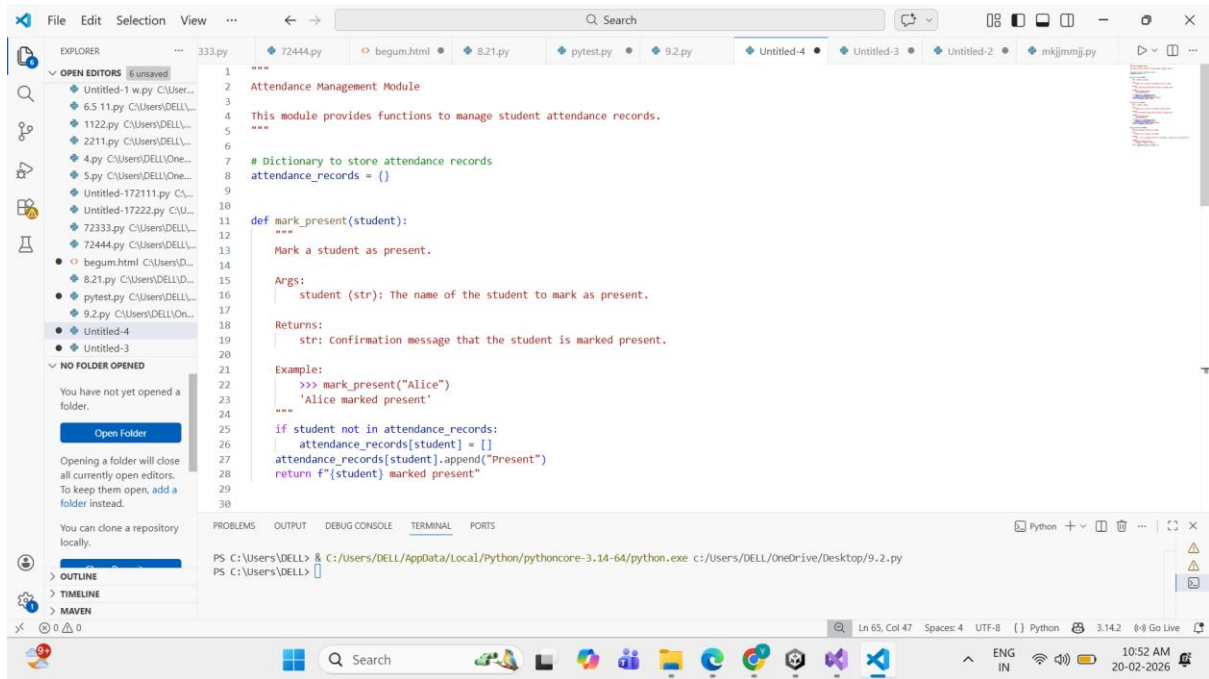


Problem 4: Attendance Management Module

Task:

1. Create a module attendance.py with functions:
 - o mark_present(student)
 - o mark_absent(student)
 - o get_attendance(student)
2. Add proper docstrings.
3. Generate and view documentation in terminal and browse

CODE:



Problem 5: File Handling Function

Consider the function:

```
def read_file(filename):
```

```
    with open(filename, 'r') as f:
```

```
        return f.read()
```

Task:

1. Write documentation using all three formats.
2. Identify which style best explains exception handling.
3. Justify your recommendation.

CODE:

